

1. S2SAMPLEPROGRAM.c

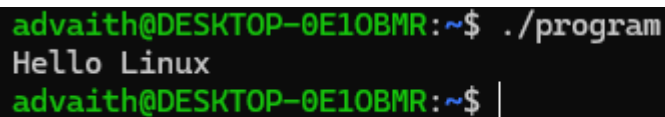
```
#include<stdio.h>
```

```
int main()
{
    printf("Hello Linux\n");
    return 0;
}
```

Compile: gcc s2sampleprogram.c -o firstprogram

./firstprogram

Output ---



```
advaith@DESKTOP-0E10BMR:~$ ./program
Hello Linux
advaith@DESKTOP-0E10BMR:~$ |
```

2. Read User Name

```
#include<stdio.h>
```

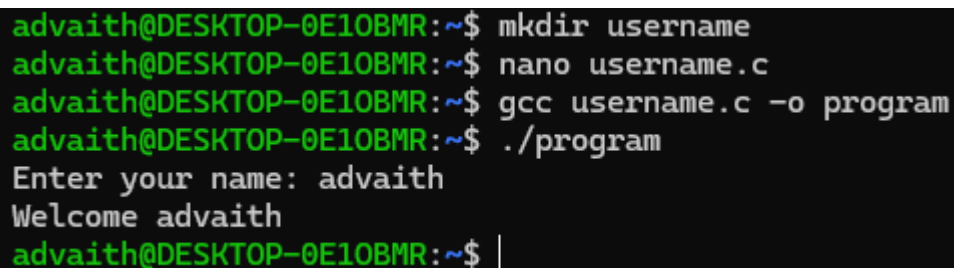
```
int main()
{
    char name[30];

    printf("Enter your name: ");
    scanf("%s",name);

    printf("Welcome %s\n",name);

    return 0;
}
```

Output—



```
advaith@DESKTOP-0E10BMR:~$ mkdir username
advaith@DESKTOP-0E10BMR:~$ nano username.c
advaith@DESKTOP-0E10BMR:~$ gcc username.c -o program
advaith@DESKTOP-0E10BMR:~$ ./program
Enter your name: advaith
Welcome advaith
advaith@DESKTOP-0E10BMR:~$ |
```

3. Display Process ID

```
#include<stdio.h>
```

```
#include<unistd.h>

int main()
{
    printf("Process ID = %d\n",getpid());

    return 0;
}
```

Output—

```
advaith@DESKTOP-0E1OBMR:~$ mkdir processid
advaith@DESKTOP-0E1OBMR:~$ nano processid.c
advaith@DESKTOP-0E1OBMR:~$ gcc processid.c -o program
advaith@DESKTOP-0E1OBMR:~$ ./program
Process ID = 720
advaith@DESKTOP-0E1OBMR:~$ |
```

4. Fork()

```
#include<stdio.h>
#include<unistd.h>

int main()
{
    fork();

    printf("Hello\n");

    return 0;
}
```

Output—

```
advaith@DESKTOP-0E1OBMR:~$ mkdir fork
advaith@DESKTOP-0E1OBMR:~$ nano fork.c
advaith@DESKTOP-0E1OBMR:~$ gcc fork.c -o program
advaith@DESKTOP-0E1OBMR:~$ ./program
Hello
Hello
```

5. Parent and Child

```
#include<stdio.h>
#include<unistd.h>

int main()
{
    pid_t pid;

    pid=fork();
```

```

    if(pid==0)
    {
        printf("I am Child Process\n");
    }
    else
    {
        printf("I am Parent Process\n");
    }

    return 0;
}

```

Output—

```

advaith@DESKTOP-0E10BMR:~$ gcc parentchild.c -o program
advaith@DESKTOP-0E10BMR:~$ ./program
I am Parent Process
I am Child Process
advaith@DESKTOP-0E10BMR:~$ |

```

6. Display parent and Child PID

```

#include<stdio.h>
#include<unistd.h>

int main()
{
    pid_t pid;

    pid=fork();

    if(pid==0)
    {
        printf("Child PID = %d\n",getpid());
        printf("Parent PID = %d\n",getppid());
    }
    else
    {
        printf("Parent PID = %d\n",getpid());
    }

    return 0;
}

```

Output--

```

advaith@DESKTOP-0E10BMR:~$ nano parentchildpid.c
advaith@DESKTOP-0E10BMR:~$ gcc parentchildpid.c -o program
advaith@DESKTOP-0E10BMR:~$ ./program
Parent PID = 908
Child PID = 909
Parent PID = 908

```

7. Wait()

```
#include<stdio.h>
#include<unistd.h>
#include<sys/wait.h>

int main()
{
    pid_t pid;

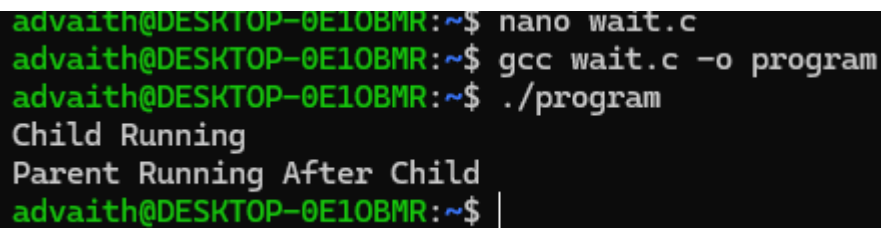
    pid=fork();

    if(pid==0)
    {
        printf("Child Running\n");
    }
    else
    {
        wait(NULL);

        printf("Parent Running After Child\n");
    }

    return 0;
}
```

Output—



```
advaith@DESKTOP-0E1OBMR:~$ nano wait.c
advaith@DESKTOP-0E1OBMR:~$ gcc wait.c -o program
advaith@DESKTOP-0E1OBMR:~$ ./program
Child Running
Parent Running After Child
advaith@DESKTOP-0E1OBMR:~$ |
```

8. Wait two()

```
#include<stdio.h>
#include<unistd.h>

int main()
{
    printf("Before exec()\n");

    execlp("ls","ls",NULL);

    printf("After exec()\n");

    return 0;
}
```

```

advaith@DESKTOP-0E10BMR:~$ gcc waittwo.c -o program
advaith@DESKTOP-0E10BMR:~$ ./program
Before exec()
ID Process fork hello parent parentchild.c parentchildpid.c processid.c username wait waittwo
ID.c advaith fork.c hello.c parentchild parentchildpid processid program username.c wait.c waittwo.c
advaith@DESKTOP-0E10BMR:~$ |

```

9. fork() + exec()

```

#include<stdio.h>
#include<unistd.h>
#include<sys/wait.h>

int main()
{
    pid_t pid;

    pid=fork();

    if(pid==0)
    {
        execlp("date","date",NULL);
    }
    else
    {
        wait(NULL);

        printf("Child Completed\n");
    }

    return 0;
}

```

```

advaith@DESKTOP-0E10BMR:~$ gcc forkexec.c -o program
advaith@DESKTOP-0E10BMR:~$ ./program
Fri Aug 7 09:12:17 UTC 2026
Child Completed
advaith@DESKTOP-0E10BMR:~$ |

```